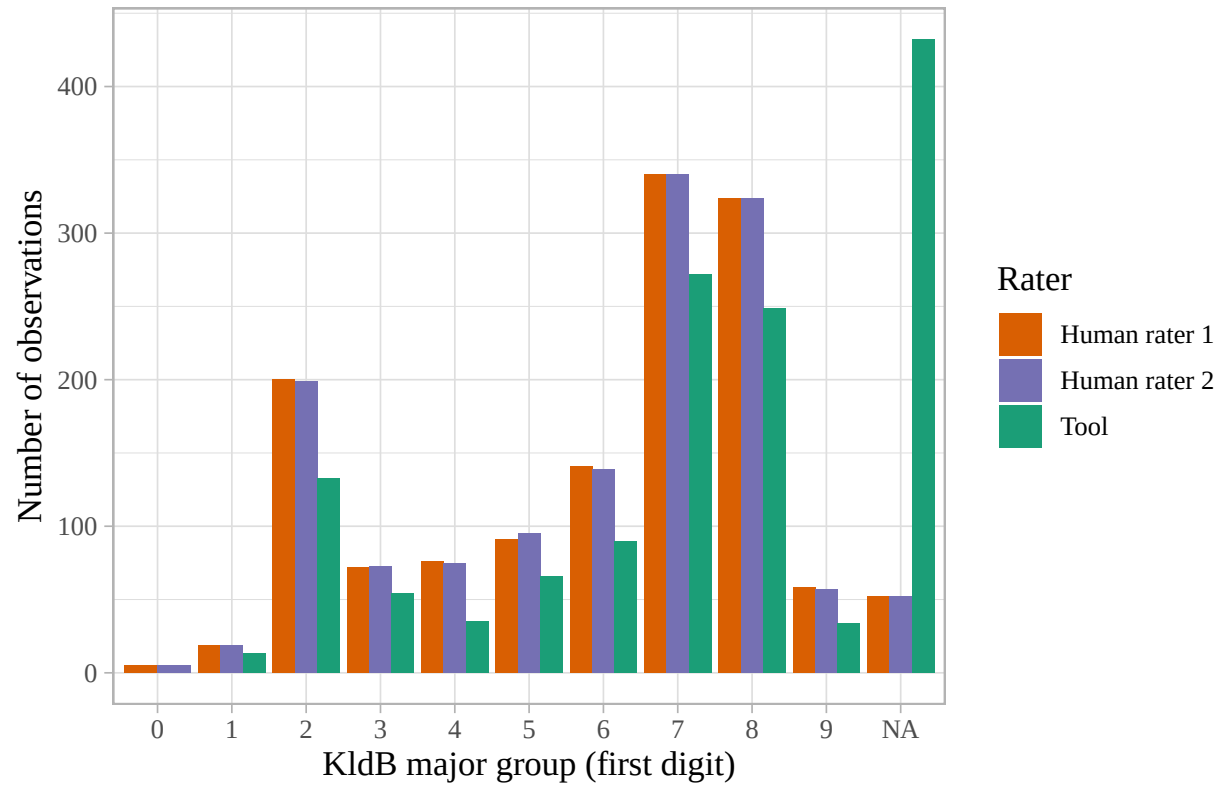
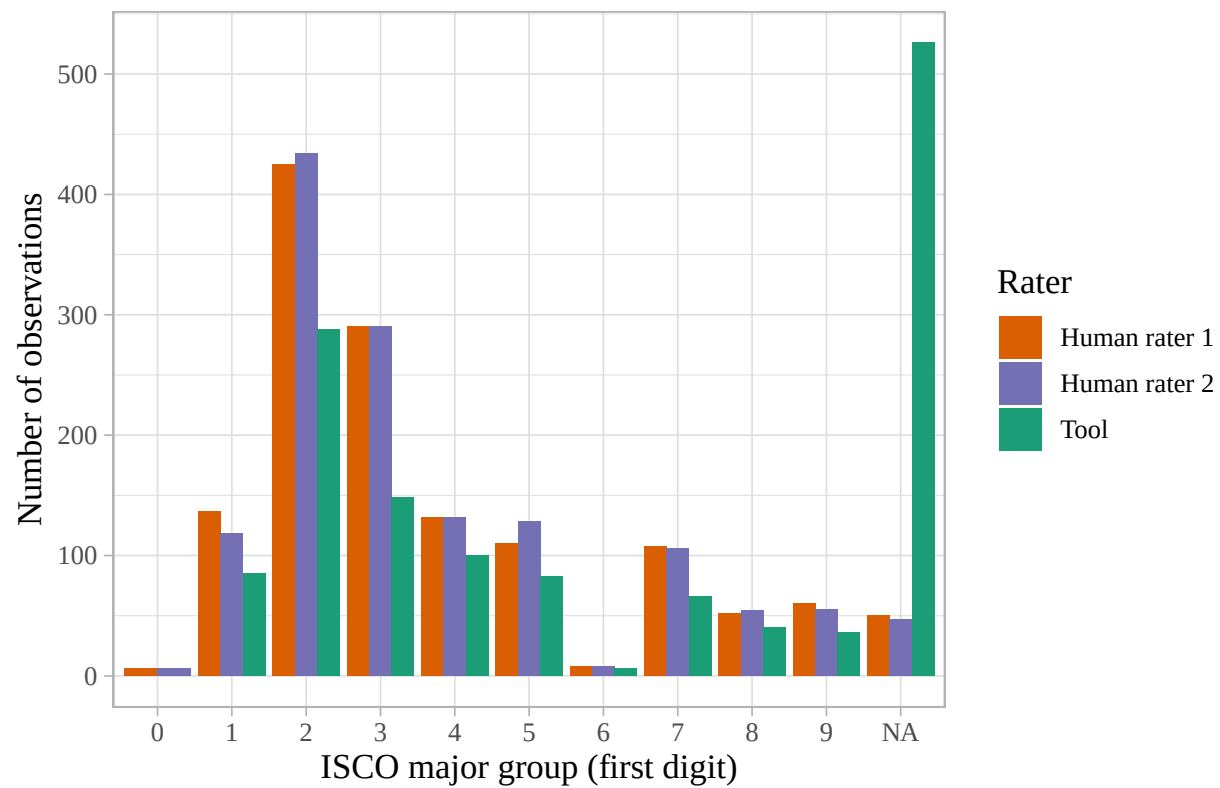


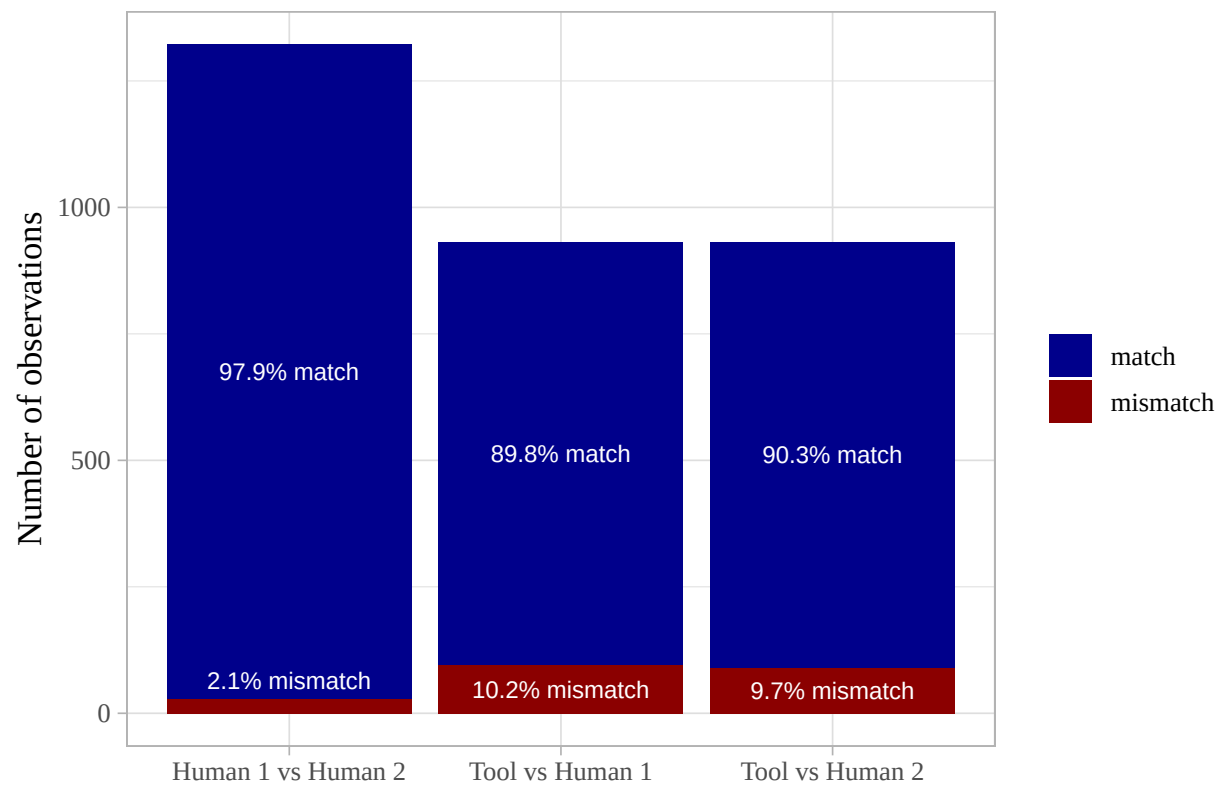
ISCO Alpha Beta Parameter Comparison

```
## [1] "C:/Users/jeroe/Uni_C/noisy_measurement/src/data_preparation.R"
```

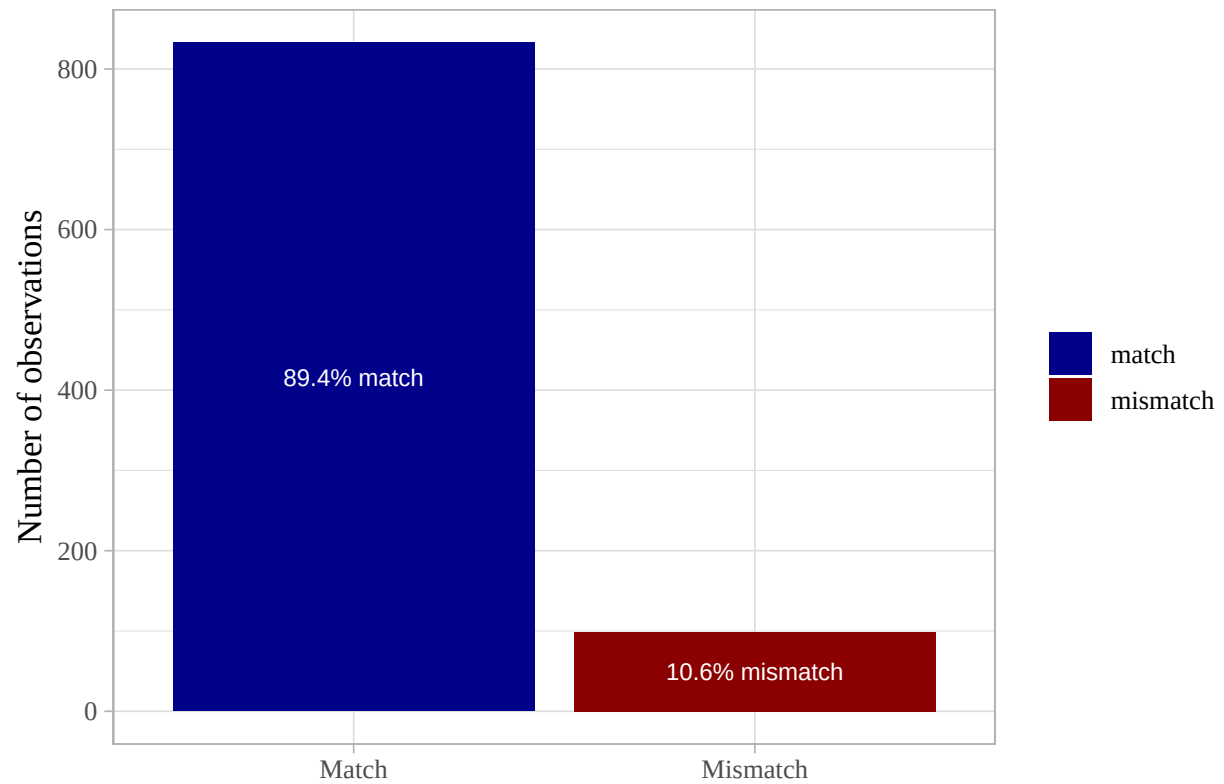




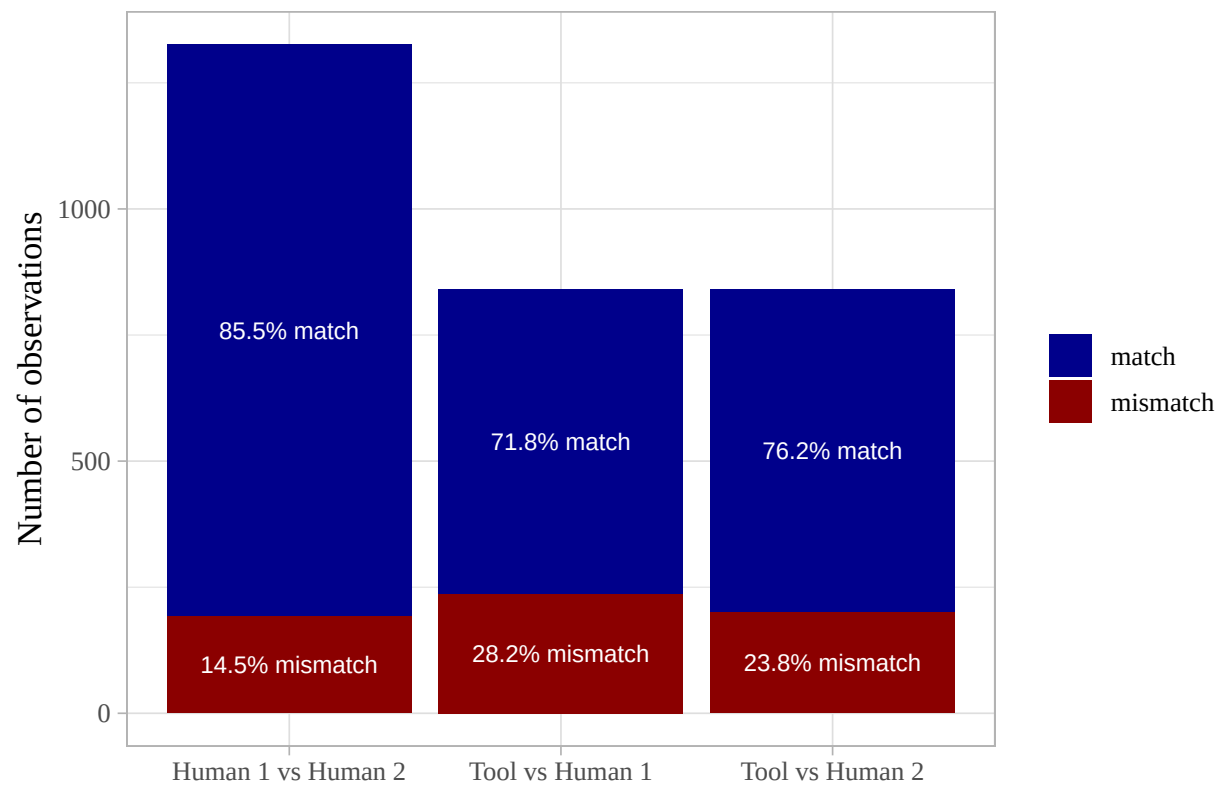
KldB: Pairwise agreement on the first digit



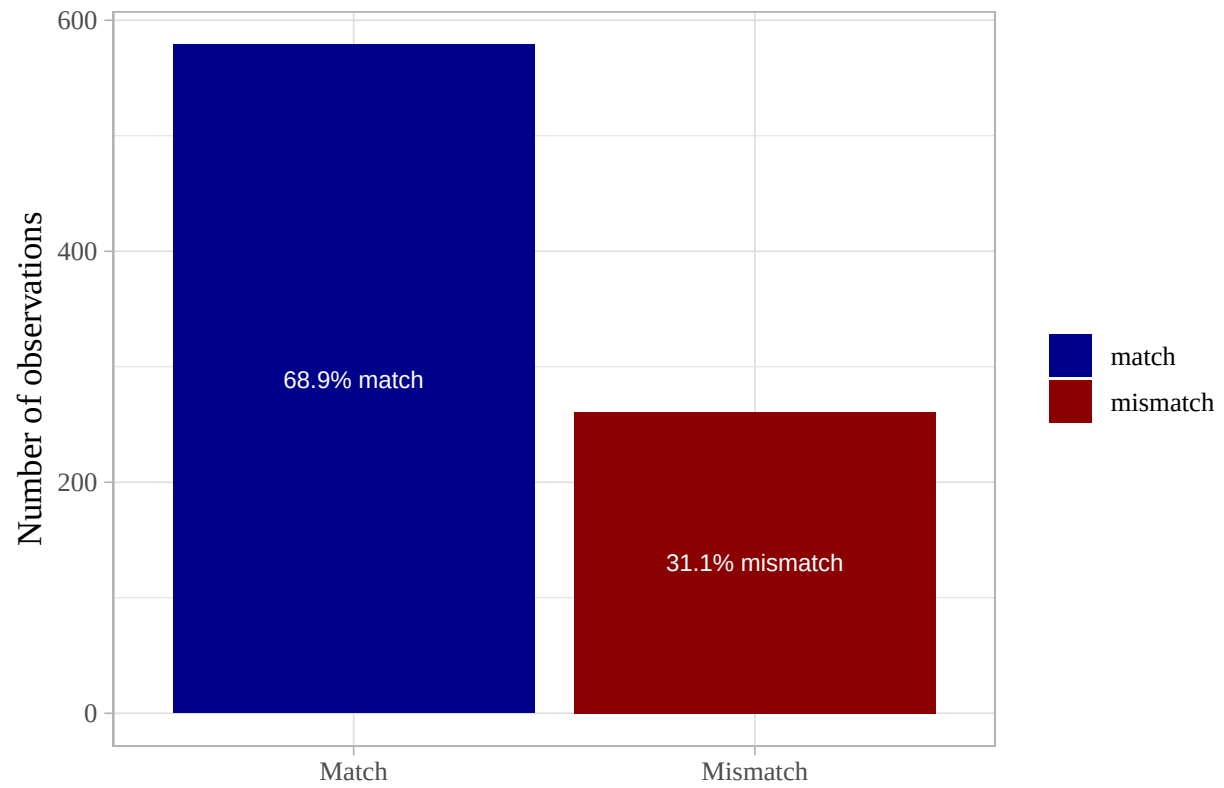
KldB: Agreement across all three sources on the first digit

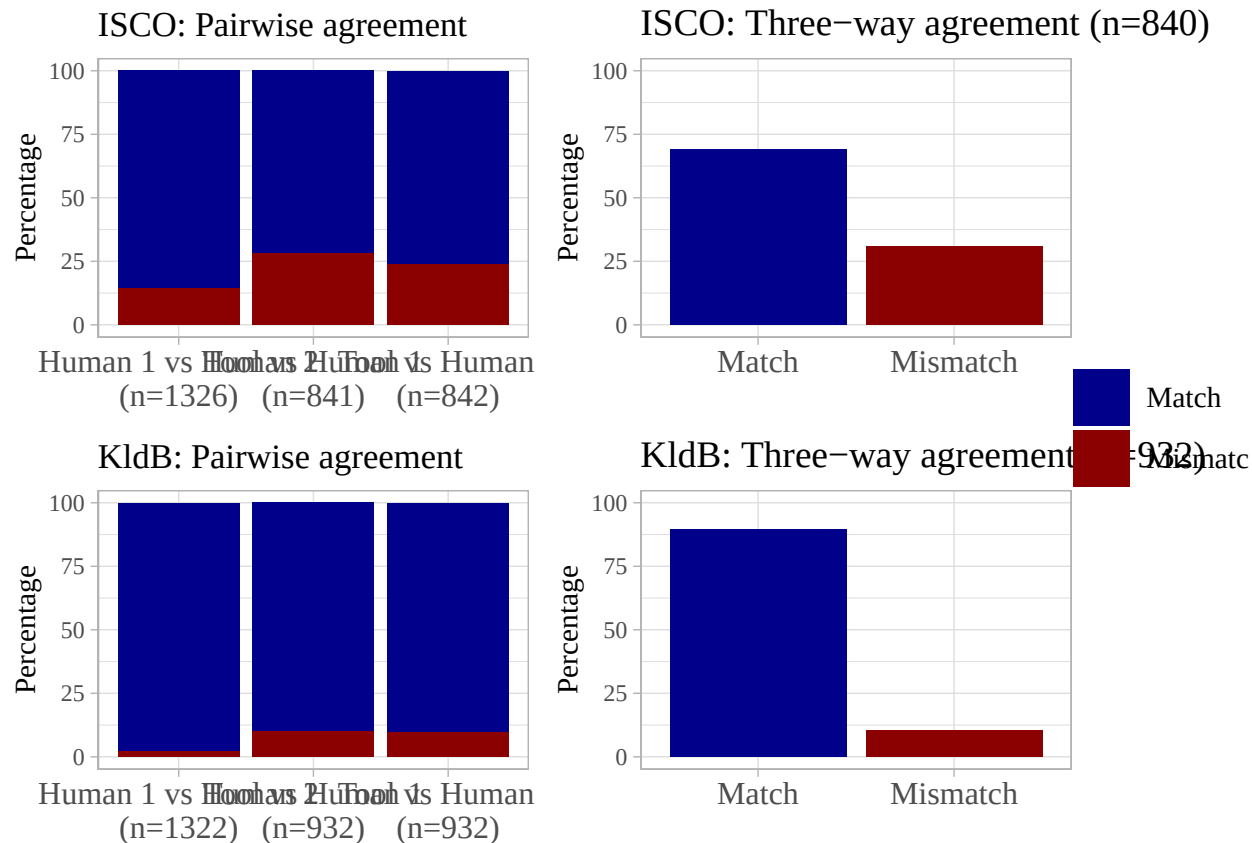


ISCO: Pairwise agreement on the first digit



ISCO: Agreement across all three sources on the first digit





```
# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)

# keep only relevant columns
isco_data <- data_basis %>%
  select(item_id, isco_tool_1st, res1_isco_1st, res2_isco_1st)

#Rename columns
setnames(
  data_basis,
  old = c("isco_tool_1st", "res1_isco_1st", "res2_isco_1st"),
  new = c("isco_tool", "isco_human1", "isco_human2")
)

head(data_basis)
```

```
##      kldb_tool_vorlaufig kldb_tool_mit_anfniveau res1_kldb res2_kldb isco_tool_vorlaufig isco_tool_mit.
##      <char>                <char>          <char>    <char>          <char>
## 1:      81883              81883      81883      81883      2433
## 2:      52112              52112      52112      52112      8322
## 3:      <NA>              <NA>       83131      83131      <NA>
```

```
## 4:          23411          23411  23411  23411          9329
## 5:          72134          72134  72132  72132          2412
## 6:          25104          25104  27224  27224          2144
##   res1_isco res2_isco kldb_infas_num          kldb_infas_str isco_infas_n
##   <char>    <char>    <char>          <char>            <cha
## 1:      2433      2433      81883          [81883] Pharmazie (s.s.T.) - Spezialist      24
## 2:      8322      8322      52112          [52112] Berufskraftfahrer(Pers./PKW.)-Fachkraft      83
## 3:      3412      3412      83131          [83131] Heilerziehungspflege, Sonderpaed. -Helfer      34
## 4:      7322      7322      23412          [23412] Drucktechnik - Fachkraft          73
## 5:      4312      4312      72132          [72132] Versicherungskaufleute - Fachkraft      43
## 6:      2149      2144      27224          [27224] Konstruktion und Geraetebau - Experte      21
##
##
## 1: [2433] Akademische und vergleichbare Fachkräfte im Bereich Vertrieb (Technik und Medizin, ohne In
## 2:                                     [8322] Personenkraftwagen-, T
## 3:                                     [3412] Nicht
## 4:
## 5:                                     [4312] Bürokräfte in der St
## 6:
##   kldb_tool_1st res1_kldb_1st res2_kldb_1st isco_tool isco_human1 isco_human2 item_id
##   <char>        <char>        <char>    <char>    <char>    <char>    <int>
## 1:           8           8           8         2         2         2         1
## 2:           5           5           5         8         8         8         2
## 3:        <NA>           8           8        <NA>         3         3         3
## 4:           2           2           2         9         7         7         4
## 5:           7           7           7         2         4         4         5
## 6:           2           2           2         2         2         2         6
```

```
#### 1) Select only ISCO, as character
```

```
isco_data <- data_basis %>%
  dplyr::select(item_id, isco_human1, isco_human2, isco_tool) %>%
  mutate(across(c(isco_human1, isco_human2, isco_tool), as.character))
```

```
### Remove all lines with NAs
```

```
isco_data <- isco_data %>%
  filter(!is.na(isco_human1) & !is.na(isco_human2) & !is.na(isco_tool))
```

```
# Keep only two categories: "2" and "3"
```

```
isco_data <- isco_data %>%
  filter(isco_human1 == "2" | isco_human1 == "3") %>%
  filter(isco_human2 == "2" | isco_human2 == "3") %>%
  filter(isco_tool == "2" | isco_tool == "3")
```

```
# 1) Number of rows where all three classify as "2"
```

```
count_all_2 <- isco_data %>%
  dplyr::filter(isco_human1 == "2", isco_human2 == "2", isco_tool == "2") %>%
  nrow()
```

```
# 2) Number of rows where all three classify as "3"
```

```
count_all_3 <- isco_data %>%
  dplyr::filter(isco_human1 == "3", isco_human2 == "3", isco_tool == "3") %>%
  nrow()
```

```
# 3) Number of rows where at least one value is "2" and at least one value is "3"
```



```

count_mixed_2_and_3 <- isco_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(c(isco_human1, isco_human2, isco_tool) == "2") &
    any(c(isco_human1, isco_human2, isco_tool) == "3")
  ) %>%
  nrow()

# 4) Number of rows where minimum one element is neither "2" nor "3" - should be 0
count_other <- isco_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(!c(isco_human1, isco_human2, isco_tool) %in% c("2", "3"))
  ) %>%
  nrow()

# Show results
count_all_2      # 235

## [1] 235
count_all_3      # 80

## [1] 80
count_mixed_2_and_3 # 42

## [1] 42
count_other      # 0

## [1] 0

# Reindex item_id (1, 2, 3,...)
isco_data <- isco_data %>%
  dplyr::mutate(item_id = dplyr::row_number())

# Collect all observed classes
isco_levels <- isco_data %>%
  dplyr::select(isco_human1, isco_human2, isco_tool) %>%
  unlist(use.names = FALSE) %>%
  unique() %>%
  sort() %>%
  as.character() %>%
  as.vector()

# K, number of distinct classes
K <- length(isco_levels)
stopifnot(K >= 2)

# Helper: map original labels to 1..K using isco_levels for stan compatibility
to_index <- function(v) as.integer(factor(v, levels = isco_levels))

# 1) Map original labels to 1..K
isco_data_idx <- isco_data %>%
  dplyr::mutate(
    isco_human1 = to_index(isco_human1),

```

```

    isco_human2 = to_index(isco_human2),
    isco_tool   = to_index(isco_tool)
  )

# 2) Long format (now without NAs)
long_data <- isco_data_idx %>%
  dplyr::transmute(item_id, r1 = isco_human1, r2 = isco_human2, r3 = isco_tool) %>%
  tidyr::pivot_longer(cols = starts_with("r"), names_to = "rater", values_to = "y") %>%
  dplyr::mutate(rater_id = as.integer(factor(rater, levels = c("r1", "r2", "r3")))) %>%
  dplyr::arrange(item_id, rater_id)

# Robust 1..I indexing
item_ids <- sort(unique(long_data$item_id))
long_data$item_id_idx <- as.integer(factor(long_data$item_id, levels = item_ids))

I <- length(item_ids) # number of items (people interviewed)
J <- 3L # number of raters
N <- nrow(long_data) # number of observations (I * J if no missing)

cat("Items after filtering:", I, "\n")

## Items after filtering: 357

cat("Observations:", N, "\n")

## Observations: 1071

cat("Classes:", K, "\n")

## Classes: 2

cat("Class levels:", paste(isco_levels, collapse = ", "), "\n")

## Class levels: 2, 3

library(dplyr)
library(tidyr)
library(ggplot2)
library(gridExtra)

# Output folder for descriptive plots of the 2 vs 3 subset
out_dir_desc <- "img/data_descriptive_plots/isco/two_categories"
dir.create(out_dir_desc, recursive = TRUE, showWarnings = FALSE)

# Color scheme consistent with your other script
agreement_palette <- c("match" = "darkblue", "mismatch" = "darkred")

# Helper: pairwise agreement (counts + mismatch percent)
pair_agreement <- function(df, a, b, label) {
  df %>%
    dplyr::filter(!is.na(.data[[a]]), !is.na(.data[[b]])) %>%
    dplyr::summarise(
      match = sum(.data[[a]] == .data[[b]]),
      mismatch = sum(.data[[a]] != .data[[b]]),
      total = dplyr::n(),
      .groups = "drop"
    )

```

```

) %>%
dplyr::mutate(
  mismatch_pct = ifelse(total > 0, 100 * mismatch / total, NA_real_),
  pair = label
)
}

# Pairwise agreements (two-way) - order matches image: Human 1 vs Human 2, Tool vs Human 1, Tool vs Human 2
isco_pairs <- dplyr::bind_rows(
  pair_agreement(isco_data, "isco_human1", "isco_human2", "Human 1 vs Human 2"),
  pair_agreement(isco_data, "isco_tool", "isco_human1", "Tool vs Human 1"),
  pair_agreement(isco_data, "isco_tool", "isco_human2", "Tool vs Human 2")
)

print(isco_pairs)

##   match mismatch total mismatch_pct      pair
## 1   337       20   357    5.602241 Human 1 vs Human 2
## 2   319       38   357   10.644258  Tool vs Human 1
## 3   331       26   357    7.282913  Tool vs Human 2

# Long format for plotting percentages
isco_pairs_long <- isco_pairs %>%
  dplyr::select(pair, match, mismatch, total, mismatch_pct) %>%
  tidyr::pivot_longer(cols = c(match, mismatch), names_to = "status", values_to = "count") %>%
  dplyr::mutate(
    status = factor(status, levels = c("mismatch", "match")),
    match_pct = 100 - mismatch_pct,
    pct = ifelse(total > 0, 100 * count / total, NA_real_),
    pair_label = paste0(pair, " (n=", total, ")")
  )

# Pairwise agreement plot - percentage only
p_isco_pairs_pct <- ggplot(isco_pairs_long, aes(x = pair_label, y = pct, fill = status)) +
  geom_col(position = "stack") +
  scale_fill_manual(values = agreement_palette, breaks = c("match", "mismatch"),
                    labels = c("Match", "Mismatch")) +
  labs(
    title = "ISCO: Pairwise agreement",
    x = NULL,
    y = "Percentage",
    fill = NULL
  ) +
  theme_light(base_size = 11, base_family = "Times") +
  theme(legend.position = "right", axis.text.x = element_text(size = 12)) +
  ylim(0, 100)

# Three-way agreement
isco_tri <- isco_data %>%
  dplyr::filter(!is.na(isco_human1), !is.na(isco_human2), !is.na(isco_tool)) %>%
  dplyr::mutate(all_three_match = (isco_human1 == isco_human2) & (isco_human2 == isco_tool)) %>%
  dplyr::summarise(
    match = sum(all_three_match),
    mismatch = sum(!all_three_match),

```

```

    total = dplyr::n(),
    .groups = "drop"
  ) %>%
  dplyr::mutate(mismatch_pct = ifelse(total > 0, 100 * mismatch / total, NA_real_))

print(isco_tri)

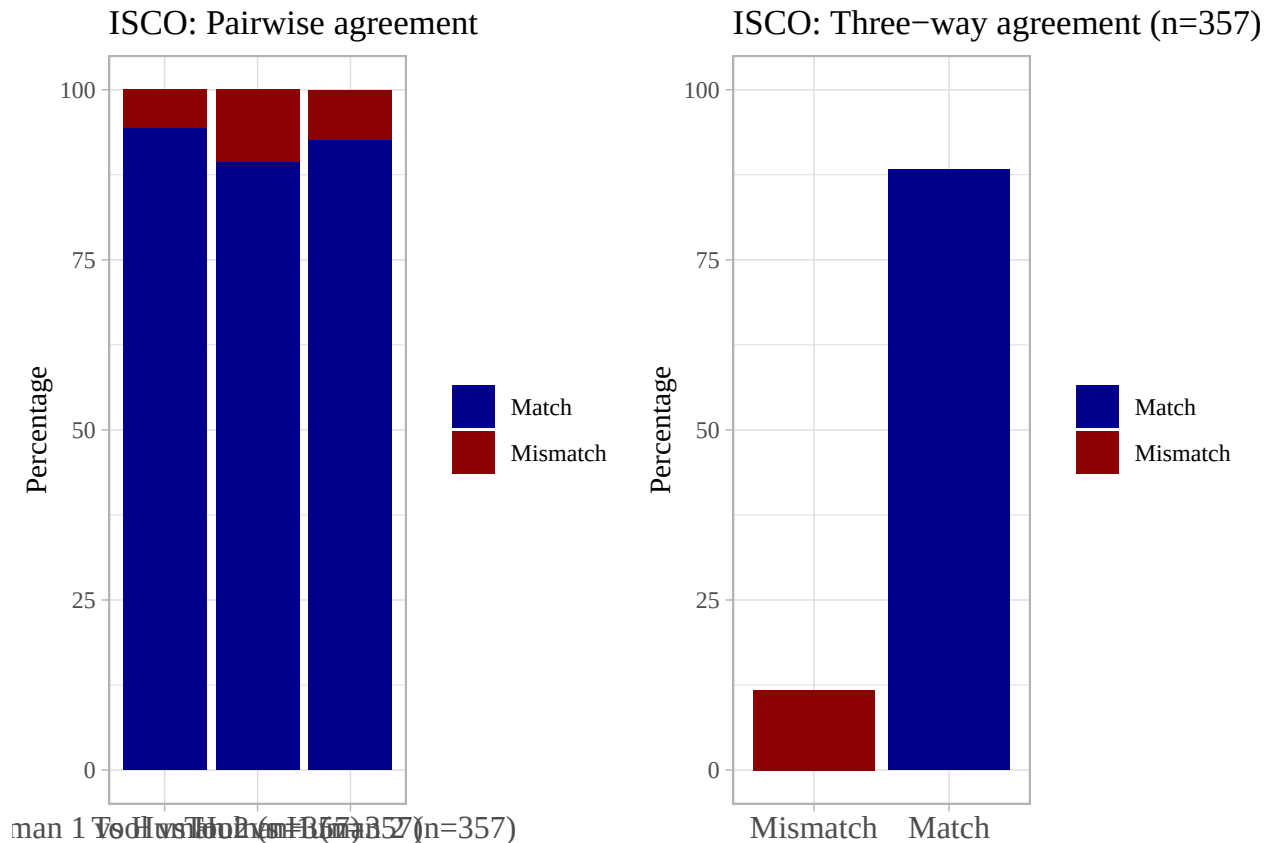
##   match mismatch total mismatch_pct
## 1   315       42   357      11.76471

isco_tri_long <- isco_tri %>%
  tidyr::pivot_longer(cols = c(match, mismatch), names_to = "status", values_to = "count") %>%
  dplyr::mutate(
    status = factor(status, levels = c("mismatch", "match")),
    match_pct = 100 - mismatch_pct,
    pct = ifelse(status == "match", match_pct, mismatch_pct),
    status_label = factor(ifelse(status == "match", "Match", "Mismatch"),
                          levels = c("Mismatch", "Match"))
  )

# Three-way agreement plot - percentage only
p_isco_tri_pct <- ggplot(isco_tri_long, aes(x = status_label, y = pct, fill = status)) +
  geom_col() +
  scale_fill_manual(values = agreement_palette, breaks = c("match", "mismatch"),
                    labels = c("Match", "Mismatch")) +
  labs(
    title = paste0("ISCO: Three-way agreement (n=", isco_tri$total[1], ")"),
    x = NULL,
    y = "Percentage",
    fill = NULL
  ) +
  theme_light(base_size = 11, base_family = "Times") +
  theme(legend.position = "right", axis.text.x = element_text(size = 12)) +
  ylim(0, 100)

# Combine plots side-by-side
p_combined <- grid.arrange(p_isco_pairs_pct, p_isco_tri_pct, ncol = 2)

```



```
# Save combined plot
ggsave(file.path(out_dir_desc, "isco_agreement_combined.png"),
  p_combined, width = 16, height = 5, dpi = 600)

# Also save individual plots
ggsave(file.path(out_dir_desc, "isco_pairwise_agreement_pct.png"),
  p_isco_pairs_pct, width = 9, height = 5, dpi = 600)
ggsave(file.path(out_dir_desc, "isco_three_way_agreement_pct.png"),
  p_isco_tri_pct, width = 7, height = 5, dpi = 600)

print(p_combined)
```

```
## TableGrob (1 x 2) "arrange": 2 grobs
##   z      cells      name      grob
## 1 1 (1-1,1-1) arrange gtable[layout]
## 2 2 (1-1,2-2) arrange gtable[layout]
```

```
# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)
```

```
# 3) Write Stan model to file
stan_file <- "dawid_skene_isco.stan"
```

```

writeLines(con = stan_file, text = '
  data {
    int<lower=2> K;           // number of categories
    int<lower=1> I;           // items
    int<lower=1> J;           // raters
    int<lower=1> N;           // number of observed ratings
    array[N] int<lower=1, upper=I> ii; // item index per observation
    array[N] int<lower=1, upper=J> jj; // rater index per observation
    array[N] int<lower=1, upper=K> y;  // observed category per observation

    vector<lower=0>[K] alpha; // prior for pi
    array[K] vector<lower=0>[K] beta; // Dirichlet priors for theta rows, vary by true class
  }

  parameters {
    simplex[K] pi; // prevalence over true classes
    array[J, K] simplex[K] theta; // confusion rows: for rater j and true class k
  }

  transformed parameters {
    // log_q_z[i, k] = log p(z_i = k | pi, theta) up to a constant
    array[I] vector[K] log_q_z;

    // Initialize with log pi
    for (i in 1:I) {
      log_q_z[i] = log(pi);
    }

    // Accumulate per observation
    for (n in 1:N) {
      int i = ii[n];
      int j = jj[n];
      int y_n = y[n];
      for (k in 1:K) {
        // Add log categorical pmf of y_n under theta[j, k, :]
        log_q_z[i, k] += categorical_lpmf(y_n | theta[j, k]);
      }
    }
  }

  model {
    // Priors
    pi ~ dirichlet(alpha);
    for (j in 1:J)
      for (k in 1:K)
        theta[j, k] ~ dirichlet(beta[k]);

    // Marginalized likelihood
    for (i in 1:I) {
      target += log_sum_exp(log_q_z[i]);
    }
  }

```

```

generated quantities {
  array[I] simplex[K] q_z;
  for (i in 1:I) {
    vector[K] lq = log_q_z[i];
    q_z[i] = softmax(lq);
  }
}
')

# Compile model once
mod <- cmdstan_model(stan_file, cpp_options = list(stan_threads = TRUE))

# Define 4 model configurations
model_configs <- list(
  list(
    name = "alpha11_beta1",
    alpha = c(1.0, 1.0),
    beta = matrix(1.0, nrow = K, ncol = K)
  )
  # list(
  #   name = "alpha11_beta5diag",
  #   alpha = c(1.0, 1.0),
  #   beta = matrix(1.0, nrow = K, ncol = K) + diag(4.0, K) # 5 on diagonal, 1 elsewhere
  # ),
  # list(
  #   name = "alpha1510_beta1",
  #   alpha = c(15.0, 10.0),
  #   beta = matrix(1.0, nrow = K, ncol = K)
  # ),
  # list(
  #   name = "alpha1510_beta5diag",
  #   alpha = c(15.0, 10.0),
  #   beta = matrix(1.0, nrow = K, ncol = K) + diag(4.0, K) # 5 on diagonal, 1 elsewhere
  # )
)

# Fit all 1 models
fits <- list()
for (config in model_configs) {
  cat("\n=== Fitting model:", config$name, "===\n")

  stan_data <- list(
    I = I,
    J = J,
    K = K,
    N = N,
    ii = as.integer(long_data$item_id_idx),
    jj = as.integer(long_data$rater_id),
    y = as.integer(long_data$y),
    alpha = config$alpha,
    beta = config$beta
  )

```

```

fit <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

fits[[config$name]] <- fit

# Check convergence
s <- fit$summary()
s$rhat <- as.numeric(s$rhat)
bad_rhat <- subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))
if (nrow(bad_rhat) > 0) {
  cat("WARNING: R-hat values above 1.1:\n")
  print(bad_rhat[, c("variable", "rhat", "ess_bulk", "ess_tail")])
} else {
  cat("No R-hat values above 1.1\n")
}
}

```

```

##
## === Fitting model: alpha11_beta1 ===
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 3 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 4 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 3 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 2 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 4 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 1 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration:  1001 / 5000 [20%] (Sampling)
## Chain 3 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 3 Iteration:  1001 / 5000 [20%] (Sampling)

```



```

## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)

```

```

## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 finished in 43.6 seconds.
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 52.5 seconds.
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 53.0 seconds.
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 62.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 52.8 seconds.
## Total execution time: 62.3 seconds.
##
## No R-hat values above 1.1
fit_basis <- fits[["alpha11_beta1"]]

if (interactive() && is.null(knitr::opts_knit$get("rmarkdown.pandoc.to"))) {
  library(shinystan)
  launch_shinystan(fit_basis)
}

library(dplyr)
library(posterior)
library(ggplot2)
library(reshape2)

out_dir <- "img/model_results/isco/two_categories"
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

```

```

# use first fit
fit_basis <- fits[["alpha11_beta1"]]

save_plot <- function(p, filename, width = 7, height = 5, dpi = 600) {
  ggsave(
    filename = file.path(out_dir, filename),
    plot = p,
    width = width,
    height = height,
    dpi = dpi
  )
}

# -----
# 0) Basic diagnostics for key parameters
# -----
diag_tbl <- fit_basis$summary(variables = c("pi", "theta")) %>%
  dplyr::select(variable, rhat, ess_bulk, ess_tail)

diag_bad <- diag_tbl %>%
  dplyr::filter(is.finite(rhat), rhat > 1.01) %>%
  dplyr::arrange(desc(rhat))

diag_bad

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# Divergences, treedepth hits, if available (cmdstanr)
div_info <- tryCatch(fit_basis$diagnostic_summary(), error = function(e) NULL)
if (is.null(div_info)) {
  div_info <- tryCatch(fit_basis$cmdstan_diagnose(), error = function(e) NULL)
}
div_info

## $num_divergent
## [1] 0 0 0 0
##
## $num_max_treedepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 0.9260143 0.9681849 0.9503473 0.9840745

# -----
# 1) Confusion matrices (theta), posterior mean and uncertainty
# -----
theta_draws_mat <- fit_basis$draws("theta", format = "draws_matrix")
theta_mean_vec <- colMeans(theta_draws_mat)

# Expectation: theta has dimensions [J rater, K true, K given]
theta_mean_arr <- array(
  theta_mean_vec,
  dim = c(J, K, K),

```

```

dimnames = list(
  rater = c("human1", "human2", "tool"),
  true = isco_levels,
  given = isco_levels
)
)

theta_human1_mean <- theta_mean_arr["human1", , ]
theta_human2_mean <- theta_mean_arr["human2", , ]
theta_tool_mean <- theta_mean_arr["tool", , ]

theta_human1_mean

##      given
## true      2      3
##    2 0.95312602 0.04687398
##    3 0.06760737 0.93239263
rowSums(theta_human1_mean)

## 2 3
## 1 1
theta_human2_mean

##      given
## true      2      3
##    2 0.98656483 0.01343517
##    3 0.02053346 0.97946654
rowSums(theta_human2_mean)

## 2 3
## 1 1
theta_tool_mean

##      given
## true      2      3
##    2 0.9528596 0.04714035
##    3 0.1295071 0.87049292
rowSums(theta_tool_mean)

## 2 3
## 1 1

plot_theta <- function(mat, title_txt) {
  long <- reshape2::melt(mat, varnames = c("True", "Given"), value.name = "Prob")
  ggplot(long, aes(x = Given, y = True, fill = Prob)) +
    geom_tile(color = "white") +
    scale_fill_gradient(low = "white", high = "darkgreen") +
    theme_light() +
    labs(
      title = title_txt,
      x = "Observed class",
      y = "True class C_i"
    ) +

```

```

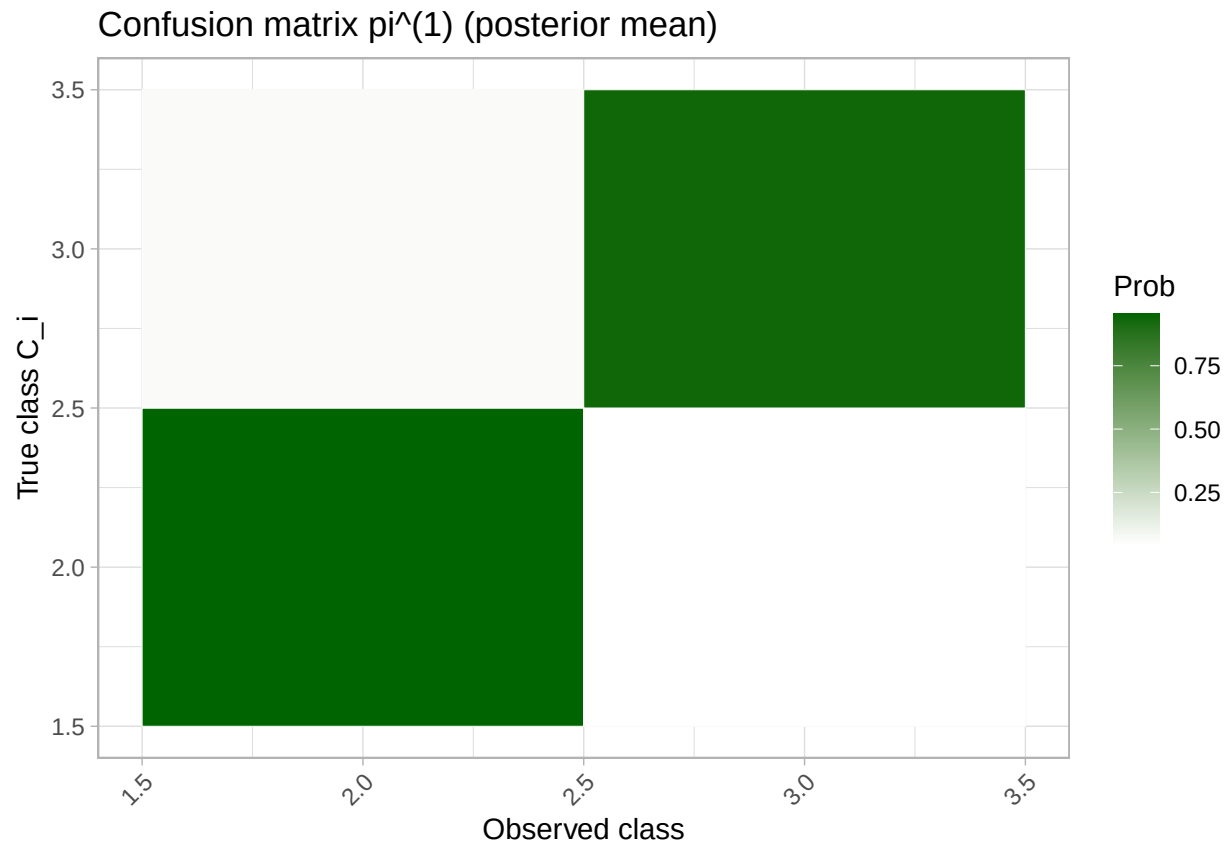
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
}

p_theta_h1_mean <- plot_theta(theta_human1_mean, "Confusion matrix  $\pi^{(1)}$  (posterior mean)")
p_theta_h2_mean <- plot_theta(theta_human2_mean, "Confusion matrix  $\pi^{(2)}$  (posterior mean)")
p_theta_tool_mean <- plot_theta(theta_tool_mean, "Confusion matrix  $\pi^{(3)}$  (posterior mean)")

save_plot(p_theta_h1_mean, "theta_human1_mean.png")
save_plot(p_theta_h2_mean, "theta_human2_mean.png")
save_plot(p_theta_tool_mean, "theta_tool_mean.png")

p_theta_h1_mean

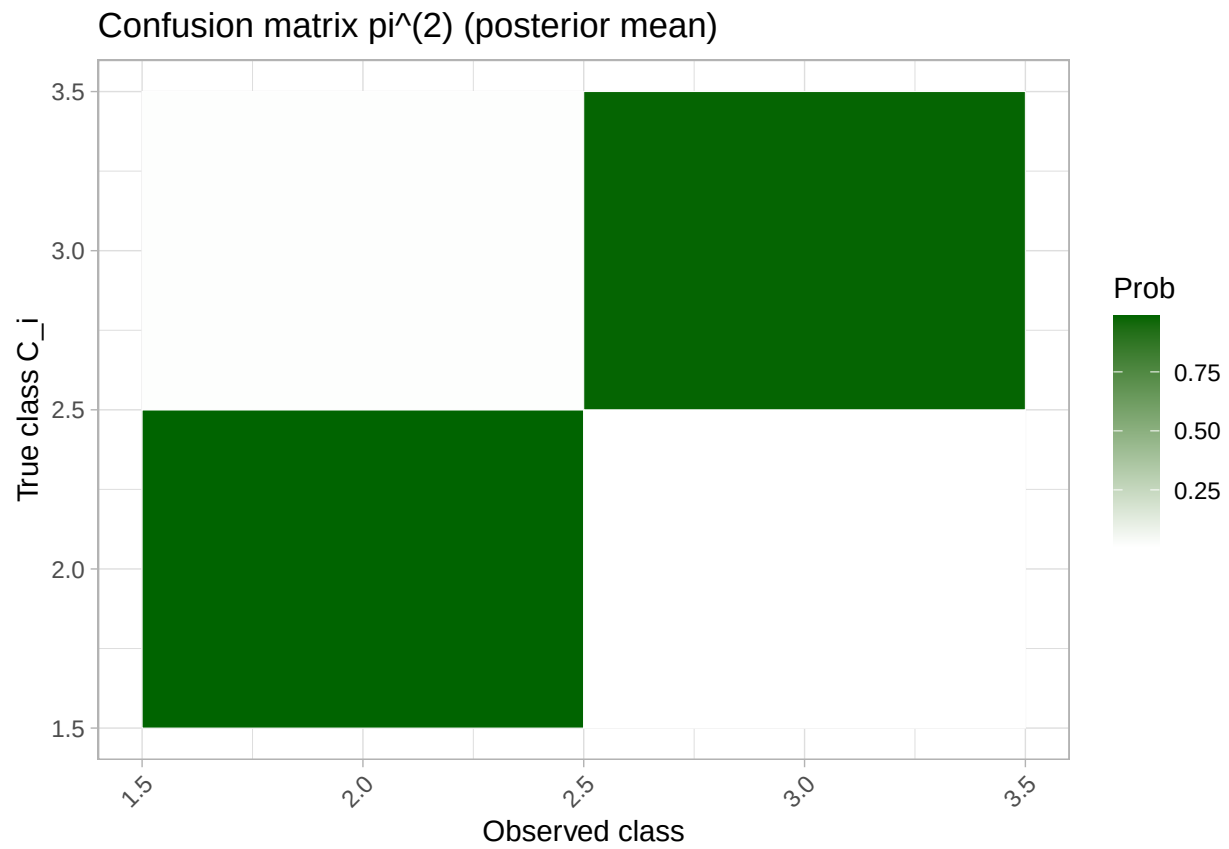
```



```

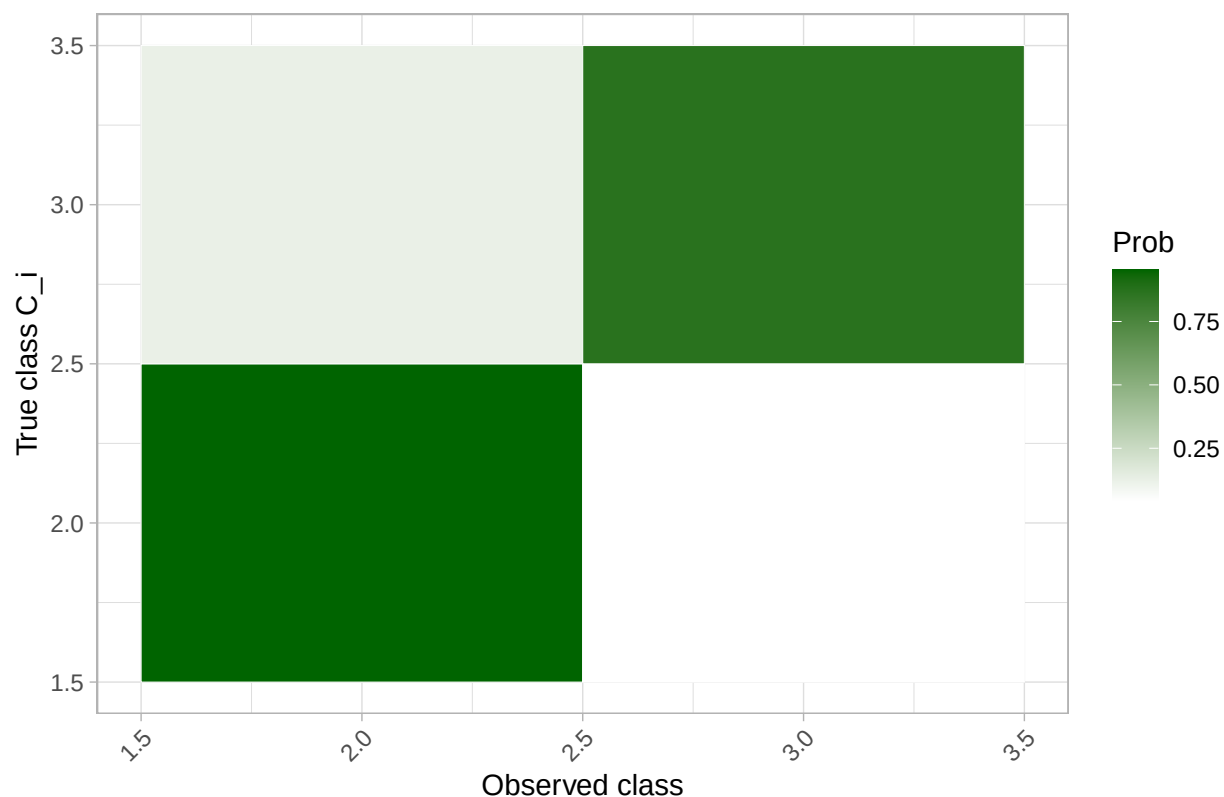
p_theta_h2_mean

```



p_theta_tool_mean

Confusion matrix $\pi^{(3)}$ (posterior mean)



```
# Credible intervals for all theta entries
theta_summ <- summarise_draws(
  as_draws_df(fit_basis$draws("theta")),
  mean, sd,
  ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
)
theta_summ
```

```
## # A tibble: 12 x 6
##   variable      mean      sd    '5%'    '50%'    '95%'
##   <chr>         <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 theta[1,1,1]  0.953  0.0135  0.929  0.954  0.973
## 2 theta[2,1,1]  0.987  0.00838  0.971  0.988  0.998
## 3 theta[3,1,1]  0.953  0.0131  0.930  0.954  0.972
## 4 theta[1,2,1]  0.0676  0.0274  0.0286  0.0646  0.118
## 5 theta[2,2,1]  0.0205  0.0166  0.00185  0.0164  0.0529
## 6 theta[3,2,1]  0.130  0.0350  0.0759  0.127  0.191
## 7 theta[1,1,2]  0.0469  0.0135  0.0268  0.0458  0.0708
## 8 theta[2,1,2]  0.0134  0.00838  0.00232  0.0121  0.0293
## 9 theta[3,1,2]  0.0471  0.0131  0.0276  0.0461  0.0702
## 10 theta[1,2,2]  0.932  0.0274  0.882  0.935  0.971
## 11 theta[2,2,2]  0.979  0.0166  0.947  0.984  0.998
## 12 theta[3,2,2]  0.870  0.0350  0.809  0.873  0.924
```

```
# Posterior SD heatmaps for theta (uncertainty by cell)
theta_sd_vec <- apply(theta_draws_mat, 2, sd)
theta_sd_arr <- array(
```

```

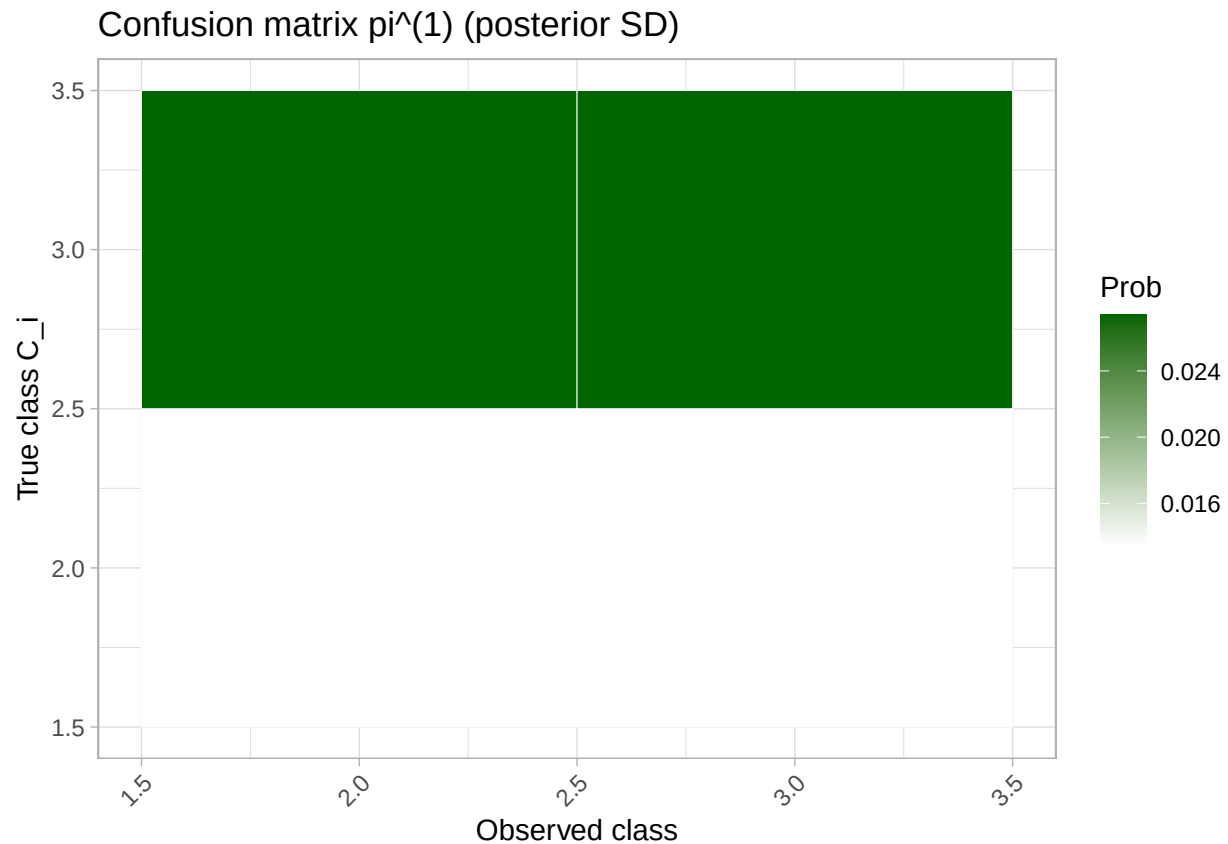
theta_sd_vec,
dim = c(J, K, K),
dimnames = list(
  rater = c("human1", "human2", "tool"),
  true = isco_levels,
  given = isco_levels
)
)

p_theta_h1_sd <- plot_theta(theta_sd_arr["human1", , ], "Confusion matrix  $\pi^{(1)}$  (posterior SD)")
p_theta_h2_sd <- plot_theta(theta_sd_arr["human2", , ], "Confusion matrix  $\pi^{(2)}$  (posterior SD)")
p_theta_tool_sd <- plot_theta(theta_sd_arr["tool", , ], "Confusion matrix  $\pi^{(3)}$  (posterior SD)")

save_plot(p_theta_h1_sd, "theta_human1_sd.png")
save_plot(p_theta_h2_sd, "theta_human2_sd.png")
save_plot(p_theta_tool_sd, "theta_tool_sd.png")

p_theta_h1_sd

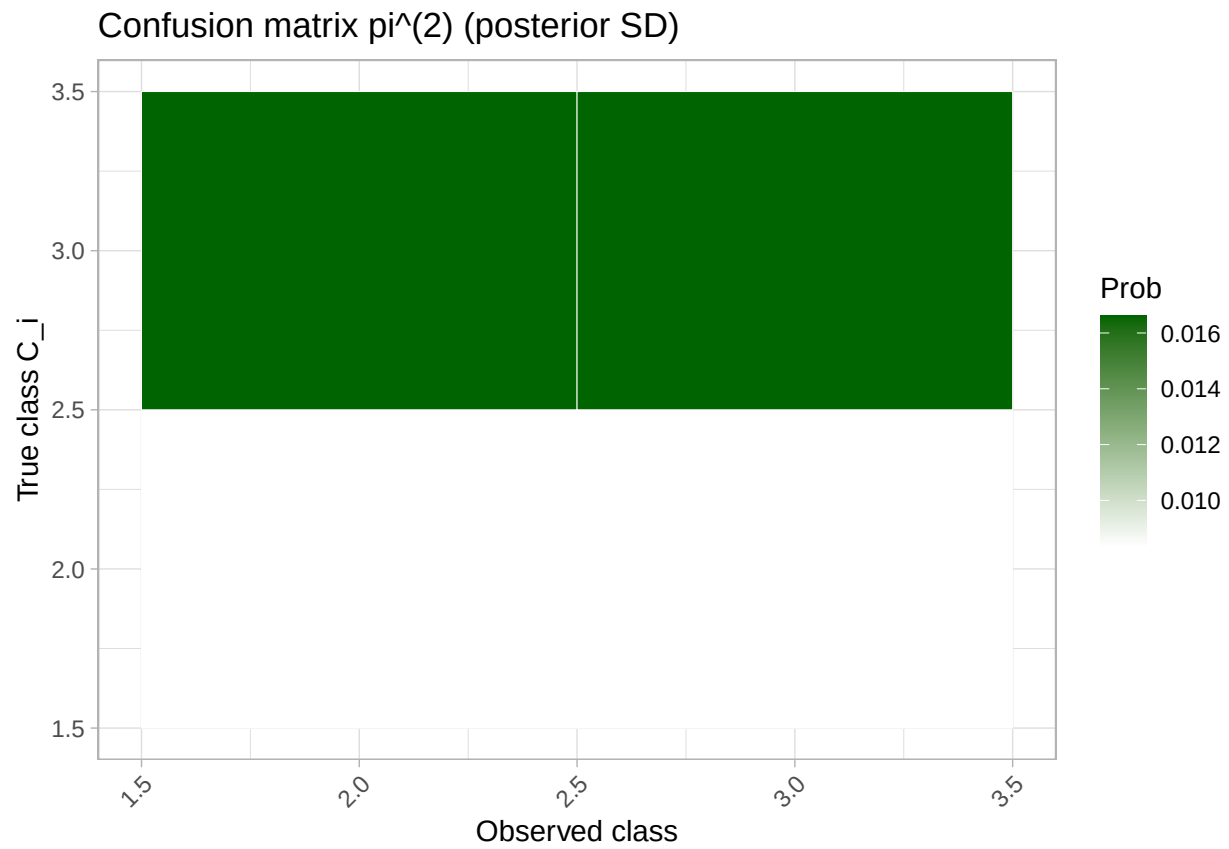
```



```

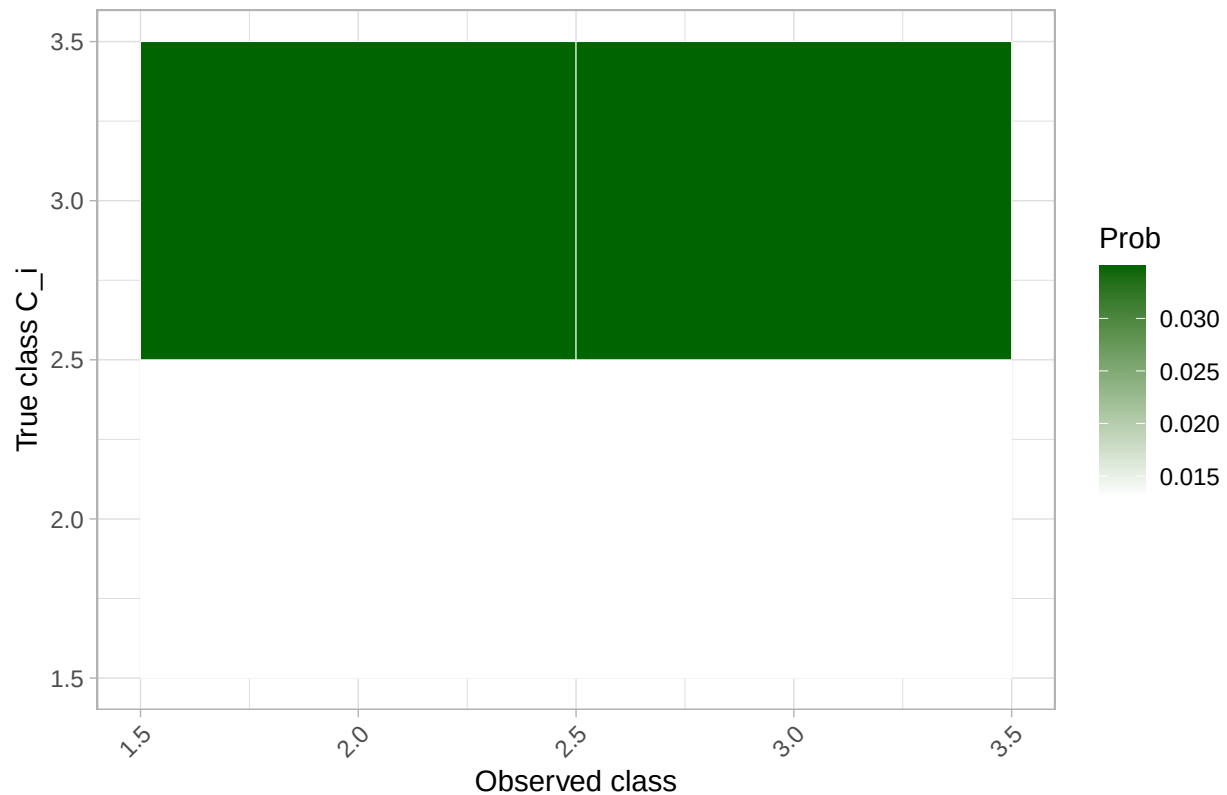
p_theta_h2_sd

```

p_theta_tool_sd

Confusion matrix $\pi^{(3)}$ (posterior SD)



```
# -----
# 2) Class prevalence ( $\pi$ ), posterior mean and uncertainty
# -----
pi_summ <- summarise_draws(
  as_draws_df(fit_basis$draws("pi")),
  mean, sd,
  ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
)
pi_summ
```

```
## # A tibble: 2 x 6
##   variable mean      sd `5%` `50%` `95%`
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 pi[1]      0.726 0.0240 0.685 0.726 0.765
## 2 pi[2]      0.274 0.0240 0.235 0.274 0.315
```

```
pi_means <- colMeans(as_draws_matrix(fit_basis$draws("pi")))
pi_df <- data.frame(
  class = isco_levels,
  prevalence = as.numeric(pi_means)
)
```

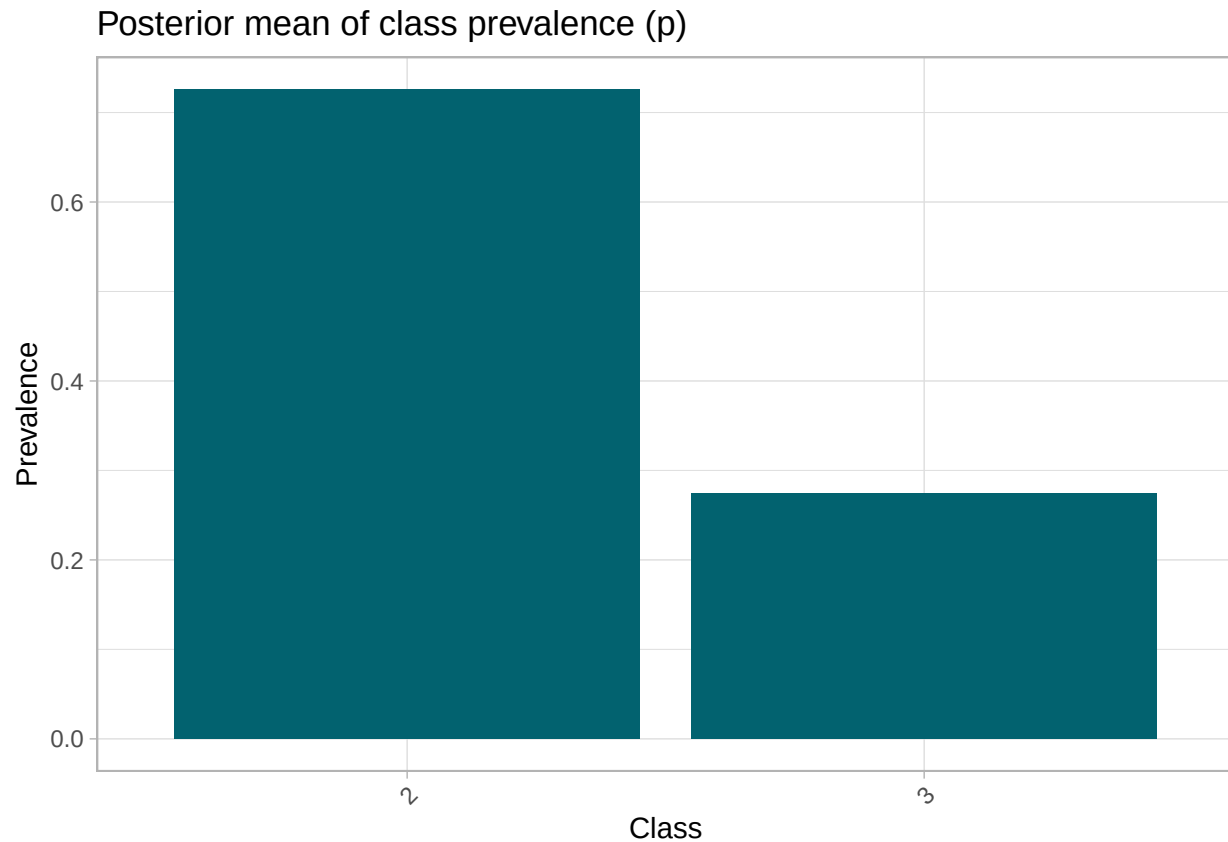
```
p_pi_bar <- ggplot(pi_df, aes(x = class, y = prevalence)) +
  geom_bar(stat = "identity", fill = "#02626f") +
  theme_light() +
  labs(
    title = "Posterior mean of class prevalence ( $p$ )",
```

```

x = "Class",
y = "Prevalence"
) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

save_plot(p_pi_bar, "pi_mean_bar.png", width = 6, height = 4)
p_pi_bar

```



```

# -----
# 2b) Rater level scalar summaries from theta
# -----

pi_mean_vec <- as.numeric(pi_means)
names(pi_mean_vec) <- isco_levels

weighted_diag_mass <- function(mat, w) sum(diag(mat) * w)
weighted_offdiag_mass <- function(mat, w) sum((1 - diag(mat)) * w)

rater_summary <- data.frame(
  rater = c("human1", "human2", "tool"),
  weighted_diag_mass = c(
    weighted_diag_mass(theta_human1_mean, pi_mean_vec),
    weighted_diag_mass(theta_human2_mean, pi_mean_vec),
    weighted_diag_mass(theta_tool_mean, pi_mean_vec)
  ),
  weighted_offdiag_mass = c(
    weighted_offdiag_mass(theta_human1_mean, pi_mean_vec),

```

```

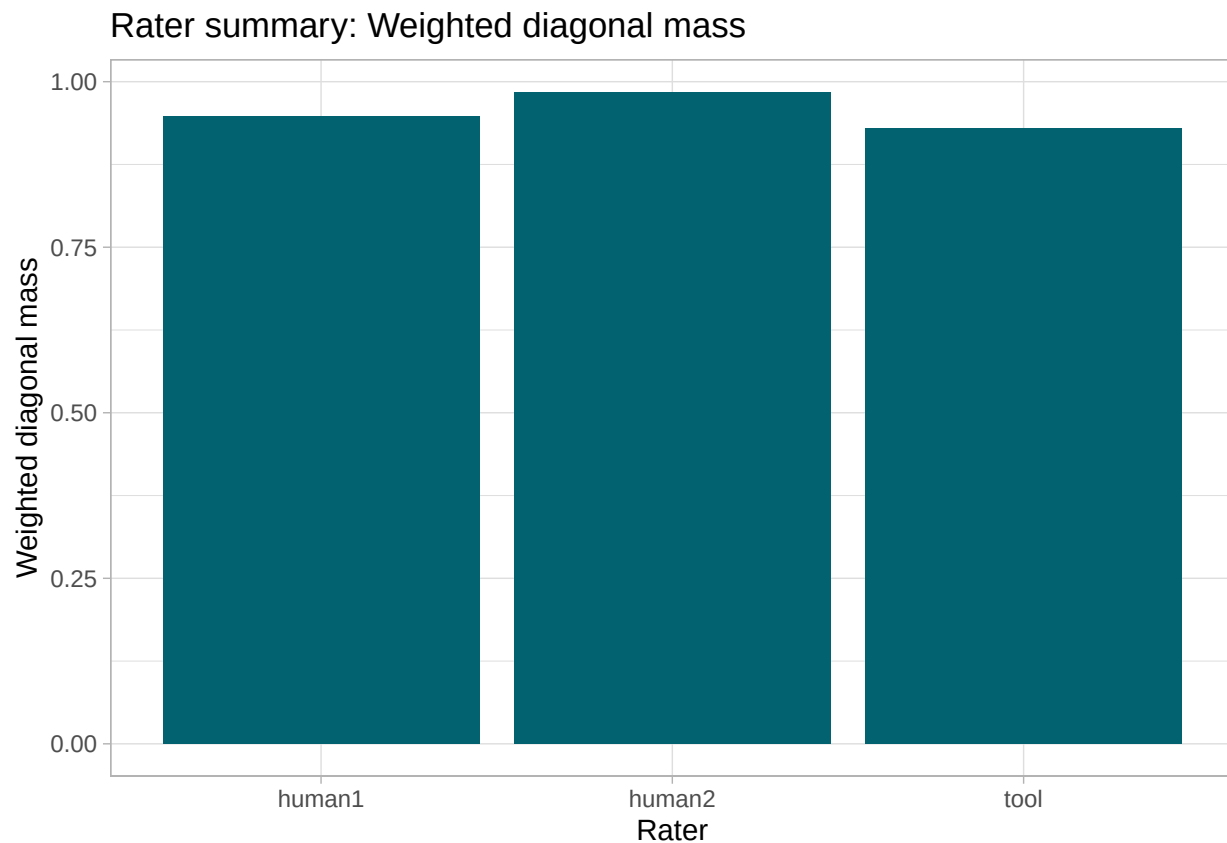
    weighted_offdiag_mass(theta_human2_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_tool_mean, pi_mean_vec)
  )
)
rater_summary

##      rater weighted_diag_mass weighted_offdiag_mass
## 1 human1      0.9474405      0.05255954
## 2 human2      0.9846183      0.01538168
## 3  tool      0.9302728      0.06972715

p_rater_diag <- ggplot(rater_summary, aes(x = rater, y = weighted_diag_mass)) +
  geom_bar(stat = "identity", fill = "#02626f") +
  theme_light() +
  labs(
    title = "Rater summary: Weighted diagonal mass",
    x = "Rater",
    y = "Weighted diagonal mass"
  )

save_plot(p_rater_diag, "rater_weighted_diag_mass.png", width = 6, height = 4)
p_rater_diag

```



```

# -----
# 3) Posterior class probabilities per item ( $q_i, j = P(C_i = j \mid data)$ )
# -----
qz_draws_mat <- fit_basis$draws("q_z", format = "draws_matrix")

```

```

qz_mean_vec <- colMeans(qz_draws_mat)

qz_mean <- matrix(
  qz_mean_vec,
  nrow = I, ncol = K,
  dimnames = list(item = item_ids, class = isco_levels)
)

qz_df <- as.data.frame(qz_mean)
p_cols <- paste0("p_", isco_levels)
colnames(qz_df) <- p_cols
qz_df$item_id <- item_ids

# MAP label
qz_df$map_class_idx <- max.col(qz_df[p_cols], ties.method = "first")
qz_df$map_class_label <- isco_levels[qz_df$map_class_idx]

# Uncertainty measures
eps <- 1e-12
qz_df$max_p <- apply(qz_df[p_cols], 1, max)

qz_df$entropy <- -rowSums(as.matrix(qz_df[p_cols]) * log(as.matrix(qz_df[p_cols]) + eps))

qz_df$margin <- apply(qz_df[p_cols], 1, function(p) {
  ps <- sort(p, decreasing = TRUE)
  ps[1] - ps[2]
})

# Credible set per item, smallest set reaching cumulative posterior mass >= level
credible_set <- function(p, classes, level = 0.95) {
  o <- order(p, decreasing = TRUE)
  cum <- cumsum(p[o])
  m <- which(cum >= level)[1]
  classes[o[seq_len(m)]]
}
qz_df$cs95 <- apply(qz_df[p_cols], 1, function(x) credible_set(x, isco_levels, level = 0.95))

table(lengths(qz_df$cs95))

##
## 1 2
## 353 4

# Plots of uncertainty and confidence
p_maxp_hist <- ggplot(qz_df, aes(x = max_p)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior max probability", y = "Count",
       title = "Model confidence per item") +
  xlim(0.6, 1.1)

p_entropy_hist <- ggplot(qz_df, aes(x = entropy)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +

```

```

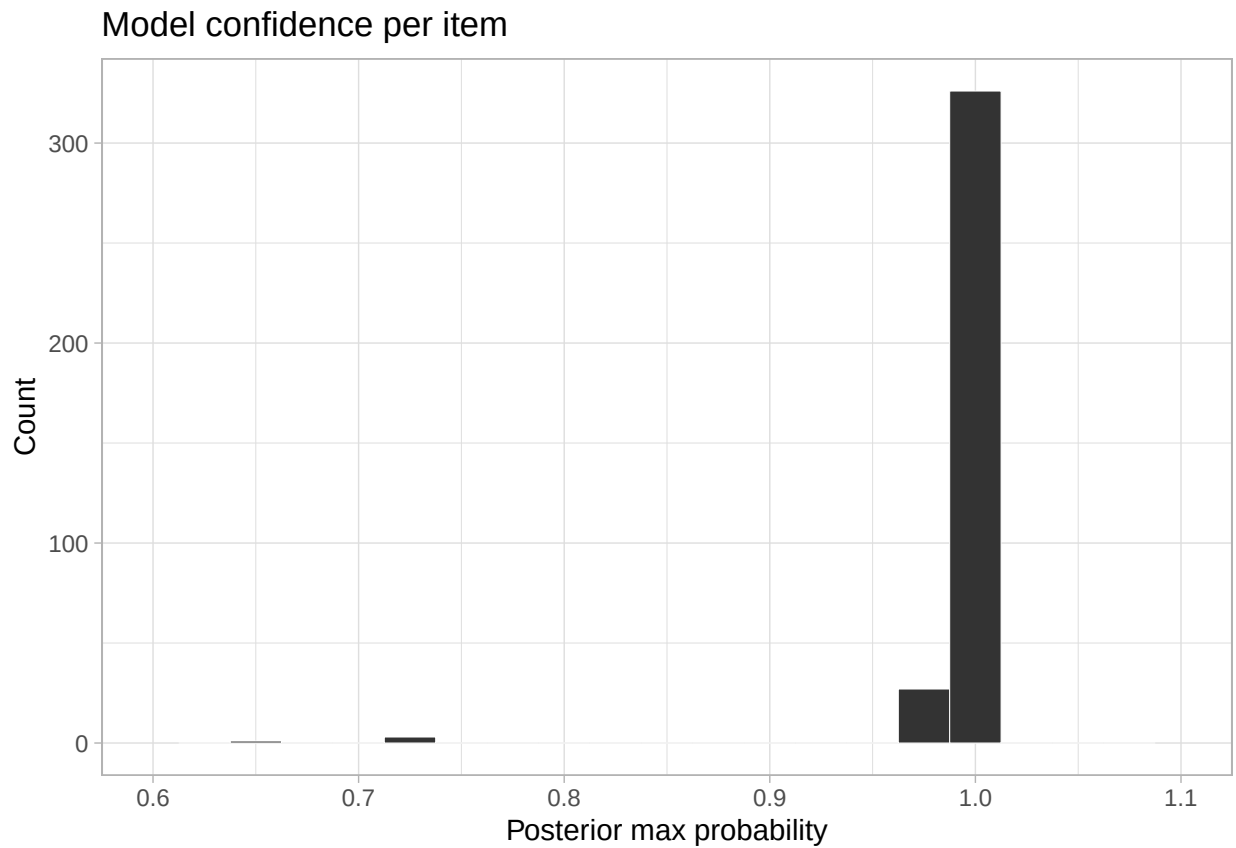
labs(x = "Posterior entropy", y = "Count",
     title = "Item uncertainty (entropy)") +
xlim(-0.1, 1.0)

p_margin_hist <- ggplot(qz_df, aes(x = margin)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior margin (top1 minus top2)", y = "Count",
       title = "Item uncertainty (margin)") +
  xlim(0, 1.1)

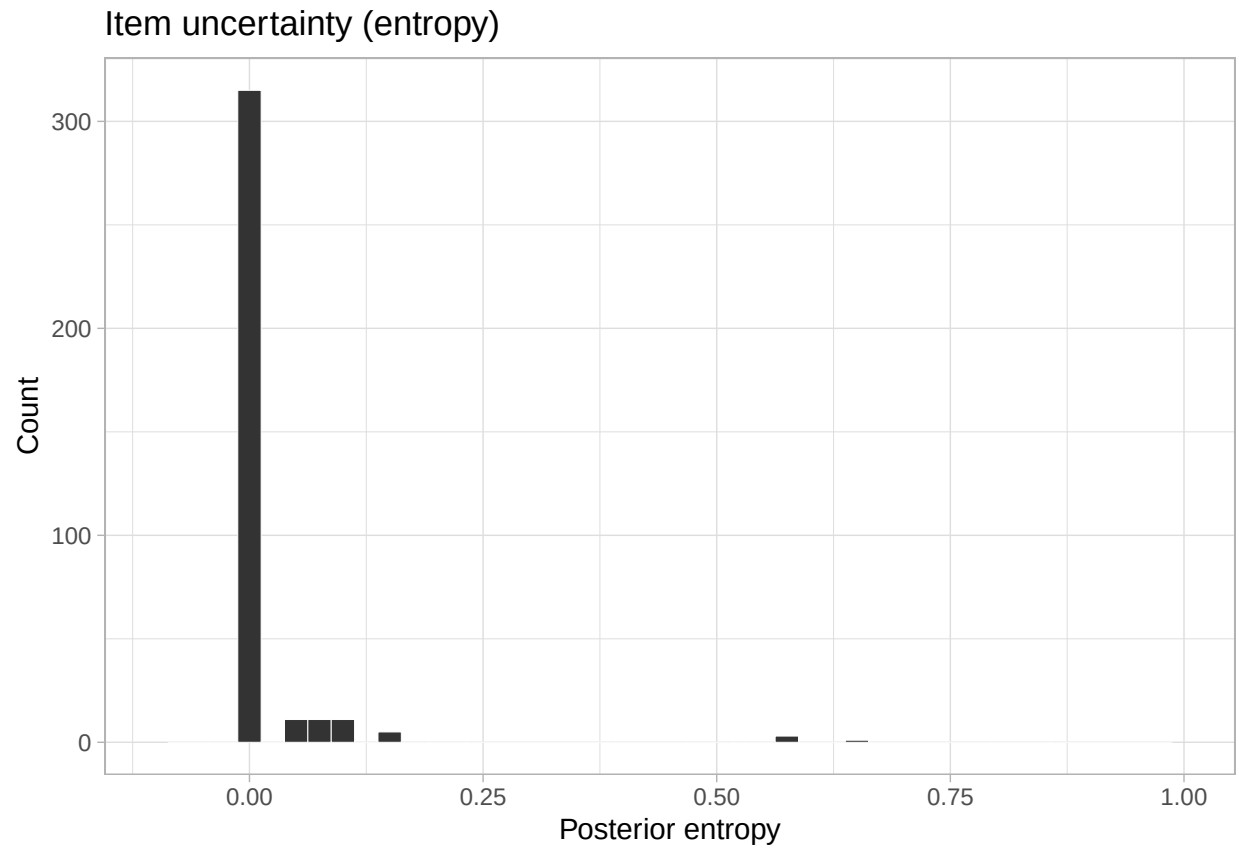
save_plot(p_maxp_hist, "qz_maxp_hist.png", width = 6, height = 4)
save_plot(p_entropy_hist, "qz_entropy_hist.png", width = 6, height = 4)
save_plot(p_margin_hist, "qz_margin_hist.png", width = 6, height = 4)

p_maxp_hist

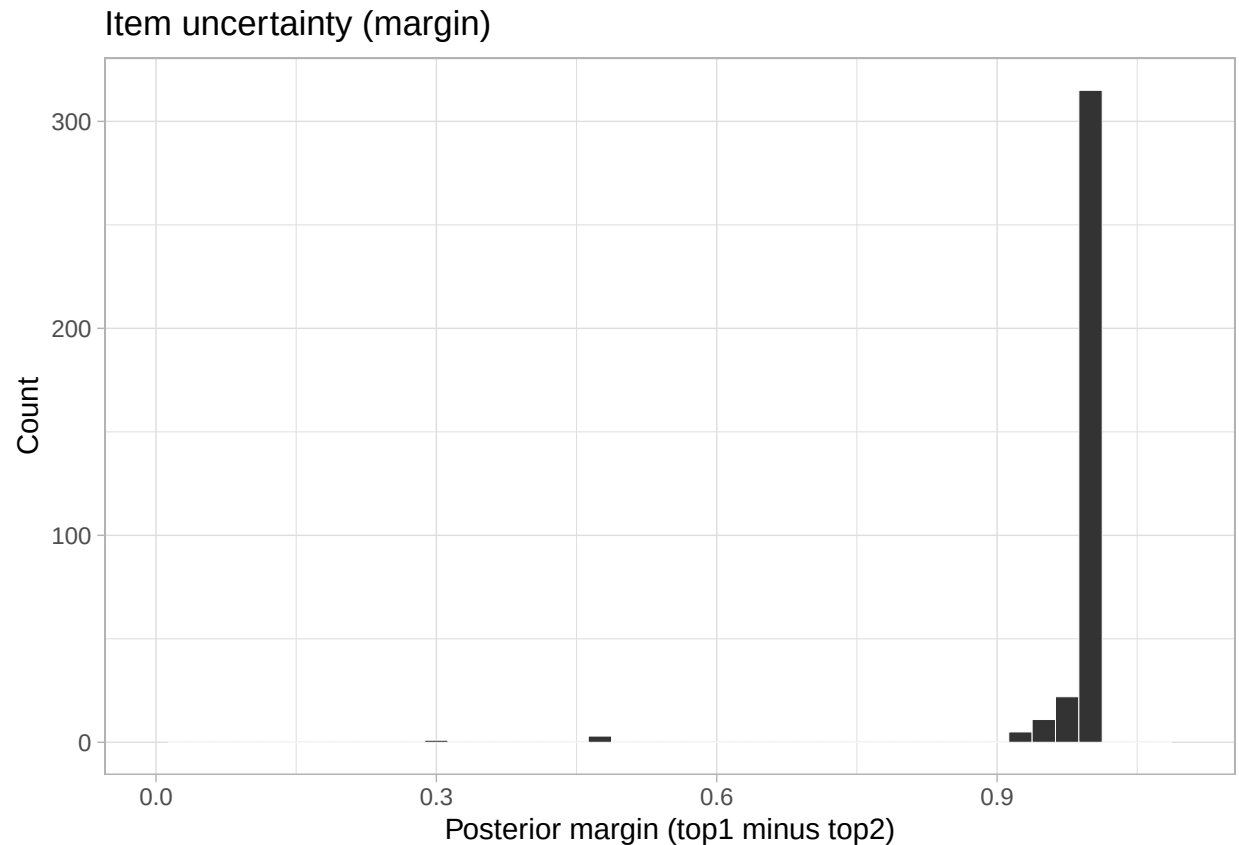
```



```
p_entropy_hist
```



p_margin_hist



```
# -----
# 3b) MAP distribution vs pi (sanity check)
# -----
map_tab <- qz_df %>%
  count(map_class_label, name = "n_map") %>%
  mutate(map_share = n_map / sum(n_map)) %>%
  rename(class = map_class_label)

pi_tab <- data.frame(class = isco_levels, pi_mean = as.numeric(pi_means))

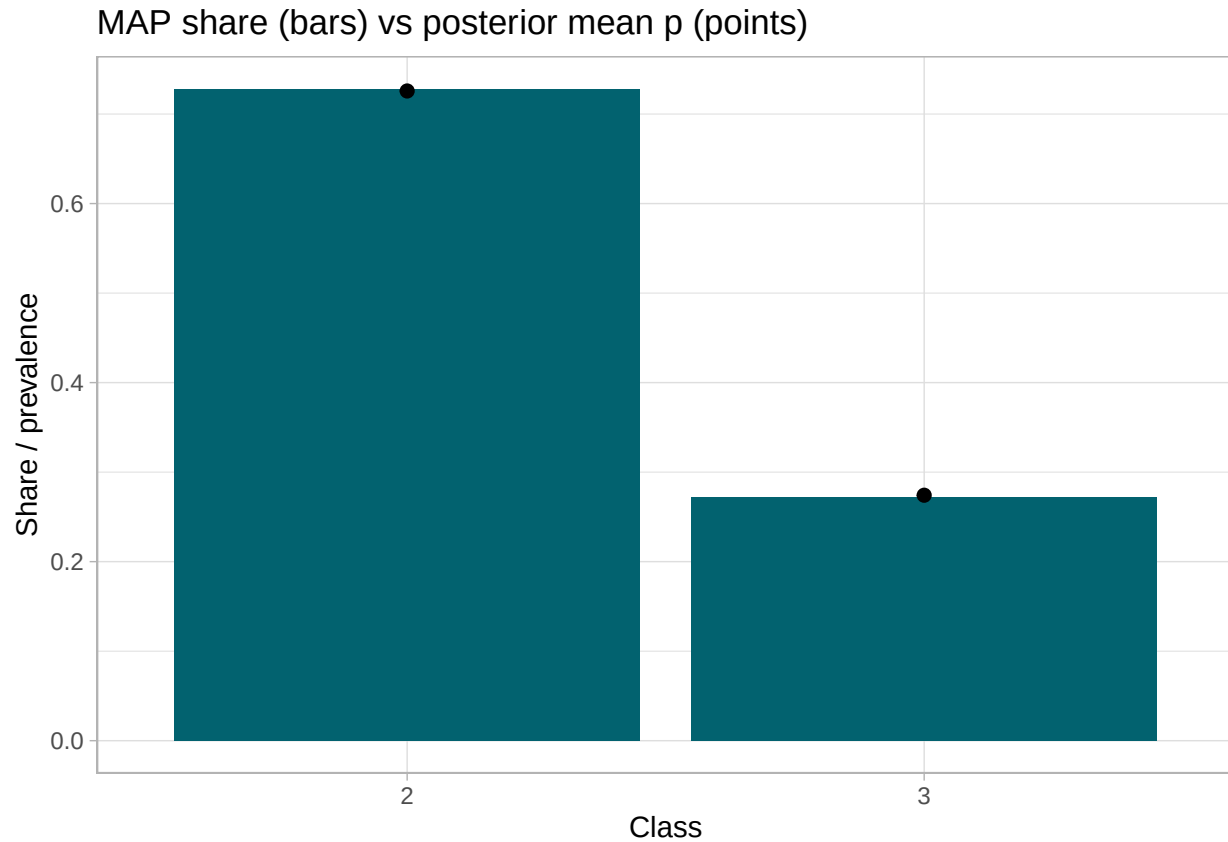
map_vs_pi <- left_join(map_tab, pi_tab, by = "class")
map_vs_pi
```

```
##   class n_map map_share  pi_mean
## 1     2    260 0.7282913 0.7257777
## 2     3     97 0.2717087 0.2742223

p_map_vs_pi <- ggplot(map_vs_pi, aes(x = class)) +
  geom_col(aes(y = map_share), fill = "#02626f") +
  geom_point(aes(y = pi_mean), size = 2) +
  theme_light() +
  labs(
    title = "MAP share (bars) vs posterior mean p (points)",
    x = "Class",
    y = "Share / prevalence"
  )
```



```
save_plot(p_map_vs_pi, "map_share_vs_pi.png", width = 6, height = 4)
p_map_vs_pi
```



```
# -----
# 4) Merge predictions with original ratings, include uncertainty
# -----
ratings <- isco_data %>%
  dplyr::select(item_id, isco_human1, isco_human2, isco_tool)

predictions <- qz_df %>%
  dplyr::select(item_id, map_class_label, all_of(p_cols), max_p, entropy, margin, cs95)

ratings_with_predictions <- ratings %>%
  dplyr::left_join(predictions, by = "item_id") %>%
  dplyr::rename(
    predicted_class = map_class_label,
    rating_human1 = isco_human1,
    rating_human2 = isco_human2,
    rating_tool = isco_tool
  ) %>%
  dplyr::arrange(item_id)

head(ratings_with_predictions, 30)
```

```
##      item_id rating_human1 rating_human2 rating_tool predicted_class      p_2      p_3      ma
##      <int>      <char>      <char>      <char>      <char>      <num>      <num>      <n
```

##	1:	1	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	2:	2	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	3:	3	2	2	3	2	0.9885818784	1.141812e-02	0.9885
##	4:	4	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	5:	5	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	6:	6	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	7:	7	2	2	3	2	0.9885818784	1.141812e-02	0.9885
##	8:	8	3	3	3	3	0.0001011121	9.998989e-01	0.9998
##	9:	9	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	10:	10	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	11:	11	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	12:	12	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	13:	13	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	14:	14	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	15:	15	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	16:	16	3	3	3	3	0.0001011121	9.998989e-01	0.9998
##	17:	17	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	18:	18	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	19:	19	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	20:	20	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	21:	21	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	22:	22	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	23:	23	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	24:	24	3	3	2	3	0.0147843301	9.852157e-01	0.9852
##	25:	25	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	26:	26	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	27:	27	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	28:	28	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	29:	29	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	30:	30	2	2	2	2	0.9999204647	7.953535e-05	0.9999
##	item_id rating_human1 rating_human2 rating_tool predicted_class					p_2	p_3	ma	
##	entropy margin cs95								
##	<num> <num> <list>								
##	1:	0.0008302909	0.9998409	2					
##	2:	0.0008302909	0.9998409	2					
##	3:	0.0624208458	0.9771638	2					
##	4:	0.0008302909	0.9998409	2					
##	5:	0.0008302909	0.9998409	2					
##	6:	0.0008302909	0.9998409	2					
##	7:	0.0624208458	0.9771638	2					
##	8:	0.0010312659	0.9997978	3					
##	9:	0.0008302909	0.9998409	2					
##	10:	0.0008302909	0.9998409	2					
##	11:	0.0008302909	0.9998409	2					
##	12:	0.0008302909	0.9998409	2					
##	13:	0.0008302909	0.9998409	2					
##	14:	0.0008302909	0.9998409	2					
##	15:	0.0008302909	0.9998409	2					
##	16:	0.0010312659	0.9997978	3					
##	17:	0.0008302909	0.9998409	2					
##	18:	0.0008302909	0.9998409	2					
##	19:	0.0008302909	0.9998409	2					
##	20:	0.0008302909	0.9998409	2					
##	21:	0.0008302909	0.9998409	2					

```
## 22: 0.0008302909 0.9998409      2
## 23: 0.0008302909 0.9998409      2
## 24: 0.0769784373 0.9704313      3
## 25: 0.0008302909 0.9998409      2
## 26: 0.0008302909 0.9998409      2
## 27: 0.0008302909 0.9998409      2
## 28: 0.0008302909 0.9998409      2
## 29: 0.0008302909 0.9998409      2
## 30: 0.0008302909 0.9998409      2
##      entropy      margin      cs95
```

```
# Most uncertain items by entropy, include posterior probabilities
most_uncertain <- ratings_with_predictions %>%
  dplyr::arrange(desc(entropy)) %>%
  dplyr::select(item_id, predicted_class, entropy, margin, max_p, all_of(p_cols),
    rating_human1, rating_human2, rating_tool)

head(most_uncertain, 20)
```

##	item_id	predicted_class	entropy	margin	max_p	p_2	p_3	rating_human1	rating
##	<int>	<char>	<num>	<num>	<num>	<num>	<num>	<char>	
## 1:	285	3	0.6471875	0.3008360	0.6504180	0.34958201	0.65041799	3	
## 2:	132	2	0.5791244	0.4682270	0.7341135	0.73411350	0.26588650	2	
## 3:	234	2	0.5791244	0.4682270	0.7341135	0.73411350	0.26588650	2	
## 4:	307	2	0.5791244	0.4682270	0.7341135	0.73411350	0.26588650	2	
## 5:	118	3	0.1473146	0.9326374	0.9663187	0.03368132	0.96631868	2	
## 6:	125	3	0.1473146	0.9326374	0.9663187	0.03368132	0.96631868	2	
## 7:	160	3	0.1473146	0.9326374	0.9663187	0.03368132	0.96631868	2	
## 8:	209	3	0.1473146	0.9326374	0.9663187	0.03368132	0.96631868	2	
## 9:	216	3	0.1473146	0.9326374	0.9663187	0.03368132	0.96631868	2	
## 10:	34	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 11:	51	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 12:	76	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 13:	82	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 14:	150	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 15:	226	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 16:	253	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 17:	264	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 18:	268	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 19:	280	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
## 20:	342	2	0.1102383	0.9536033	0.9768016	0.97680163	0.02319837	3	
##	item_id	predicted_class	entropy	margin	max_p	p_2	p_3	rating_human1	rating
##	rating_tool								
##	<char>								
## 1:	3								
## 2:	2								
## 3:	2								
## 4:	2								
## 5:	3								
## 6:	3								
## 7:	3								
## 8:	3								
## 9:	3								
## 10:	2								
## 11:	2								

```

## 12:      2
## 13:      2
## 14:      2
## 15:      2
## 16:      2
## 17:      2
## 18:      2
## 19:      2
## 20:      2
##      rating_tool

# -----
# 5) Observed agreement, descriptive, not model based
# -----
mean(isco_data$isco_human1 == isco_data$isco_tool, na.rm = TRUE)

## [1] 0.8935574

mean(isco_data$isco_human2 == isco_data$isco_tool, na.rm = TRUE)

## [1] 0.9271709

isco_data %>%
  dplyr::count(isco_human1, isco_human2, isco_tool) %>%
  dplyr::arrange(desc(n))

##      isco_human1 isco_human2 isco_tool      n
##      <char>      <char>      <char> <int>
## 1:           2           2           2    235
## 2:           3           3           3     80
## 3:           2           2           3     11
## 4:           3           2           2     11
## 5:           3           3           2     11
## 6:           2           3           3      5
## 7:           2           3           2      3
## 8:           3           2           3      1

# -----
# Posterior intervals: diagonal entries + "sharpness" by rater
# (includes Tool as rater 3)
# -----
library(posterior)

# 1) Pull theta draws once
# Column names are typically like: theta[<rater>,<>true>,<obs>]
theta_mat <- fit_basis$draws("theta", format = "draws_matrix")

# 2) Helper: extract one theta cell as a numeric vector of draws
get_theta_draws <- function(rater_id, true_class_idx, obs_class_idx) {
  nm <- sprintf("theta[%d,%d,%d]", rater_id, true_class_idx, obs_class_idx)
  if (!nm %in% colnames(theta_mat)) {
    stop("Column not found in draws matrix: ", nm,
         "\nCheck theta indexing and parameter naming in Stan output.")
  }
  as.numeric(theta_mat[, nm])
}

```

```

# 3) Helper: posterior summary (mean, sd, 5/50/95 percentiles)
summ <- function(x) {
  c(
    mean = mean(x),
    sd   = sd(x),
    q05  = unname(quantile(x, 0.05)),
    q50  = unname(quantile(x, 0.50)),
    q95  = unname(quantile(x, 0.95))
  )
}

# Binary case: class index 1 corresponds to "2", class index 2 corresponds to "3"
# (because isco_levels is sorted and then mapped to 1..K in your preparation code)
# rater_id: 1 = human1, 2 = human2, 3 = tool

# 4) Diagonal draws for each rater
h1_22 <- get_theta_draws(1, 1, 1)
h1_33 <- get_theta_draws(1, 2, 2)

h2_22 <- get_theta_draws(2, 1, 1)
h2_33 <- get_theta_draws(2, 2, 2)

t_22  <- get_theta_draws(3, 1, 1)
t_33  <- get_theta_draws(3, 2, 2)

# 5) "Sharpness" score per rater (simple average of diagonals in the 2-class case)
s1 <- 0.5 * (h1_22 + h1_33)
s2 <- 0.5 * (h2_22 + h2_33)
s3 <- 0.5 * (t_22  + t_33)

# 6) Differences in sharpness (who is sharper than whom)
d12 <- s1 - s2
d13 <- s1 - s3
d23 <- s2 - s3

# 7) Combine into one summary table
diag_tbl <- rbind(
  human1_true2_obs2 = summ(h1_22),
  human1_true3_obs3 = summ(h1_33),
  sharp_human1      = summ(s1),

  human2_true2_obs2 = summ(h2_22),
  human2_true3_obs3 = summ(h2_33),
  sharp_human2      = summ(s2),

  tool_true2_obs2   = summ(t_22),
  tool_true3_obs3   = summ(t_33),
  sharp_tool        = summ(s3),

  diff_h1_minus_h2  = summ(d12),
  diff_h1_minus_tool = summ(d13),
  diff_h2_minus_tool = summ(d23)
)

```

```
print(round(diag_tbl, 3))
```

##	mean	sd	q05	q50	q95
## human1_true2_obs2	0.953	0.014	0.929	0.954	0.973
## human1_true3_obs3	0.932	0.027	0.882	0.935	0.971
## sharp_human1	0.943	0.015	0.915	0.944	0.965
## human2_true2_obs2	0.987	0.008	0.971	0.988	0.998
## human2_true3_obs3	0.979	0.017	0.947	0.984	0.998
## sharp_human2	0.983	0.009	0.965	0.985	0.995
## tool_true2_obs2	0.953	0.013	0.930	0.954	0.972
## tool_true3_obs3	0.870	0.035	0.809	0.873	0.924
## sharp_tool	0.912	0.019	0.879	0.913	0.940
## diff_h1_minus_h2	-0.040	0.018	-0.072	-0.039	-0.011
## diff_h1_minus_tool	0.031	0.024	-0.007	0.031	0.070
## diff_h2_minus_tool	0.071	0.021	0.038	0.070	0.107